

📖 تاسكات مشروع: نظام إدارة المكتبة (Backend)

الهدف: بناء ال Backend الكامل للمشروع باستخدام Node.js, Express.js, MongoDB, Mongoose
تخيل نفسك المسؤول الوحيد عن ال Backend في شركة، و هتنفذ المشروع Module بعد Module بشكل احترافي.

🌱 Phase 0: إعداد المشروع (Project Setup)

Task 0.1: تجهيز بيئة العمل

- ☐ عمل `npm init` وتثبيت الحزم الأساسية (express, mongoose, dotenv, nodemon)
- ☐ تنظيم هيكل المجلدات (MVC): `config`, `middlewares`, `controllers`, `routes`, `models`, `utils`

💡 تلميح: افصل من أول يوم بين المسؤوليات (Separation of Concerns) عشان المشروع يبقى قابل للتوسع لاحقًا، مكتتبش كل حاجة في ملف واحد.

Task 0.2: الاتصال بقاعدة البيانات

- ☐ إعداد الاتصال بـ MongoDB عن طريق Mongoose
- ☐ استخدام `env.` لتخزين بيانات الاتصال الحساسة

💡 تلميح: استخدم مكتبة `dotenv` ولا تعمل commit لملف `env.` أبدًا، ضيفه في `.gitignore.`

Task 0.3: إعداد السيرفر الأساسي

- ☐ إنشاء `app.js` أو `server.js` وتشغيل السيرفر
- ☐ إضافة middleware أساسي زي `express.json()`

💡 تلميح: اعمل route بسيط زي `health/` يرجع `status: OK` عشان تتأكد إن السيرفر شغال قبل ما تكمل.

📖 Phase 1: إدارة الكتب (Books Management)

Task 1.1: عمل Book Model

☐ الحقول: `name`, `author`, `category`, `publisher`, `price`, `availableCopies`, `image`, `description`

💡 تلميح: استخدم `enum` لو التصنيفات محدودة، واستخدم `required` و `min` للتأكد من صحة البيانات (مثلاً السعر ميقاش سالب).

Task 1.2: عمل CRUD للكتب

☐ Create, Read (all + single), Update, Delete

💡 تلميح: افصل ال logic في Controller وسيب ال Route نضيف بس بيربط ال endpoint بالدالة المناسبة.

Task 1.3: البحث والتصنيف

☐ البحث بالاسم أو المؤلف

- ☐ الفلتر حسب التصنيف أو دار النشر
- ☐ الترتيب (حسب السعر مثلاً) وال Pagination

💡 تلميح: استخدم Query Params زي `=search=&category=&sort=&page=&limit?` وابني ال query بشكل ديناميكي باستخدام `(RegExp)` للبحث الجزئي.

Task 1.4: رفع صورة الكتاب

- ☐ استخدام `(multer)` لرفع صورة وتخزين اسمها/رابطها في الداتايز

💡 تلميح: خزن الصور في مجلد `(uploads)` وارجع الرابط بشكل نسبي، ومتنساش تتأكد من نوع وحجم الملف قبل الرفع.

👥 Phase 2: إدارة العملاء (Customers Management)

Task 2.1: عمل User/Customer Model

- ☐ الحقول: name, email, password, phone, type (branch / online), role

💡 تلميح: استخدم `(bcrypt)` لتشفير الباسورد قبل الحفظ باستخدام Mongoose middleware زي `.pre('save')`.

Task 2.2: التسجيل وتسجيل الدخول (Auth)

- ☐ Register / Login باستخدام JWT
- ☐ Middleware للتحقق من التوكن (Protect Routes)

💡 تلميح: خزن ال JWT Secret في `(env)`، واعمل middleware باسم `(authMiddleware)` يتحقق من التوكن قبل أي route محمي.

Task 2.3: صلاحيات المستخدمين (Roles)

- ☐ تفرقة بين: مدير - موظف - عميل
- ☐ Middleware للتحقق من الدور (Authorization)

💡 تلميح: اعمل middleware اسمه `(restrictTo('admin', 'employee'))` يقبل الأدوار المسموح لها ك parameters، ده هيوفر لك تكرار كثير.

Task 2.4: بيانات وسجل العملاء

- ☐ عرض بيانات العميل وتحديثها
- ☐ عرض سجل الطلبات السابقة الخاصة بالعميل

💡 تلميح: استخدم `(populate())` لو عندك ربط بين الطلبات والعميل، عشان تجيب بيانات مفصلة في استعلام واحد.

📄 Phase 3: المبيعات داخل الفرع (In-Branch Sales)

Task 3.1: عمل Invoice Model

- ☐ الحقول: customer, items (book + quantity + price), totalAmount, paymentMethod, employee, date

💡 تلميح: احسب (totalAmount) في ال Backend نفسه ومتعتمدش على القيمة الجاية من الفرونت إند، عشان تحمي البيانات من التلاعب.

Task 3.2: إنشاء فاتورة بيع

- ☐ إضافة كتب وكميات للفاتورة
- ☐ التأكد إن الكمية المطلوبة متوفرة في المخزون قبل البيع

💡 تلميح: استخدم MongoDB Transactions ((session)) عشان تضمن إن عملية خصم المخزون وإنشاء الفاتورة يحصلوا مع بعض أو ملخصش أي حاجة.

Task 3.3: تحديث المخزون تلقائيًا بعد البيع

- ☐ خصم الكمية المباعة من (availableCopies)

💡 تلميح: اربط الخطوة دي بـ Task 6 (إدارة المخزون) وسجل حركة "خروج" لكل عملية بيع.

🛒 Phase 4: المتجر الإلكتروني (Online Store)

Task 4.1: تصفح الكتب وعرض التفاصيل

- ☐ لعرض الكتب المتاحة فقط (Public) عامة Endpoints

💡 تلميح: فلتّر الكتب اللي (availableCopies > 0) بس تظهر للعميل الأونلاين، أو أظهر حالة "غير متوفر" بدل ما تخفيها خالص.

Task 4.2: سلة المشتريات (Cart)

- ☐ للسلة (مرتبطة بالعميل) Model
- ☐ إضافة/حذف/تعديل كمية عنصر في السلة

💡 تلميح: ممكن تخزن السلة كمصفوفة items جوه المستخدم، أو Model منفصل مرتبط بـ userId - جرّب الطريقة الأسهل الأول وطورها بعدين.

Task 4.3: إتمام الطلب (Checkout)

- ☐ تحويل محتوى السلة إلى Order
- ☐ التحقق من توفر الكمية قبل تأكيد الطلب

💡 تلميح: بعد إتمام الطلب بنجاح، فضّي السلة تلقائيًا (([] = cart.items)).

📦 Phase 5: إدارة الطلبات (Orders Management)

Task 5.1: عمل Order Model

- ☐ الحقول: customer, items, totalAmount, status, shippingAddress, createdAt
- ☐ status كـ enum: (جديد, قيد التجهيز, جاهز للشحن, تم الشحن, تم التسليم)

💡 تلميح: خلي ال status عبارة عن enum ثابت في ال Schema عشان تمنع أي قيمة غلط تتسجل.

Task 5.2: تتبع حالة الطلب

- ☐ للعميل يشوف حالة طلبه Endpoint
- ☐ للموظف/المدير لتغيير الحالة Endpoint

💡 تلميح: امنع تغيير الحالة بترتيب عشوائي (متسمحش تنقل من "جديد" لـ "تم التسليم" على طول)،
اعمل تحقق منطقي بسيط على الترتيب.

Task 5.3: عرض الطلبات السابقة للعميل

❑ Endpoint يرجع كل طلبات العميل الحالي فقط

💡 تلميح: استخدم بيانات المستخدم اللي جايه من الـ `((req.user.id))` JWT بدل ما تاخذ الـ id من الفرونٲ، عشان محدش يقدر يشوف طلبات غيره.

📊 Phase 6: إدارة المخزون (Inventory Management)

Task 6.1: عمل Stock Movement Model

❑ الحقول: `book, type (in/out), quantity, reason, date`

💡 تلميح: كل عملية بيع أو إضافة كتب جديدة لازم تسجل "حركة" في الموديل ده، ده هيبني لك تاريخ كامل لحركة المخزون.

Task 6.2: تصنيف حالة المخزون

❑ Endpoint يرجع: الكتب المتوفرة / قليلة المخزون / نافذة Endpoint

💡 تلميح: حدد `threshold` ثابت (مثلاً أقل من 5 نسخ = "قليل") وممكن تخليه رقم قابل للتعديل من إعدادات المكتبة.

Task 6.3: تقرير حركة المخزون

❑ Endpoint يعرض كل حركات الدخول والخروج لكتاب معين

💡 تلميح: استخدم `(populate('book'))` عشان تجيب اسم الكتاب مع كل حركة بدل ما تكتفي بالـ ID بس.

👤 Phase 7: الموظفين والصلاحيات (Employees & Roles)

Task 7.1: إدارة الموظفين (Admin Only)

❑ CRUD للموظفين، متاح للمدير فقط

💡 تلميح: استخدم نفس الـ `middleware` اللي عملته في `((restrictTo('admin')))` Task 2.3 بدل ما تكتب تحقق جديد.

Task 7.2: تحديد الصلاحيات لكل Endpoint

❑ مراجعة كل الـ `routes` وتحديد مين يقدر يوصلها `(admin / employee / customer)`

💡 تلميح: عمل جدول بسيط (Excel أو Markdown) يوضح كل `route` ومين مسموحله، ده هيسهل عليك المراجعة والاختبار.

📈 Phase 8: التقارير والإحصائيات (Reports & Analytics)

Task 8.1: تقرير المبيعات

❑ إجمالي المبيعات، المبيعات اليومية والشهرية

💡 تلميح: استخدم MongoDB Aggregation Pipeline (`group`, `$sum`, `$match`) بالتاريخ) بدل ما تجيب كل البيانات وتحسبها في Node.js.

Task 8.2: أكثر الكتب مبيعًا والكتب المطلوب إعادة توفيرها

- ☐ Endpoint (Top Selling) للأكثر مبيعًا
- ☐ Endpoint للكتب التي وصلت لحد أدنى من المخزون

💡 تلميح: استخدم (`sort`) و (`limit`) بعد ال (`group`) في ال Aggregation عشان تجيب أعلى N كتاب مبيعًا بسهولة.

Task 8.3: إحصائيات عامة (Dashboard)

☐ عدد العملاء، عدد الطلبات الأونلاين، قيمة المخزون الكلية

💡 تلميح: اعمل Endpoint واحد اسمه (`dashboard/stats/`) يرجع كل الأرقام دي مجمعة في response واحد، أسهل للفرونت إند.

📖 Phase 9: إدارة بيانات المكتبة (Library Settings)

Task 9.1: عمل Library Settings Model

☐ الحقول: اسم المكتبة، الهاتف، العنوان، بيانات التواصل، الشعار، إعدادات البيع

💡 تلميح: عادة الإعدادات بتبقى Document واحد بس في الكوليكشن (مش متعدد)، خليك حريص إنك تعمل update مش create كل مرة.

Task 9.2: Endpoint (Admin Only) لعرض وتحديث الإعدادات

☐ GET / PUT للإعدادات العامة

💡 تلميح: استخدم (`findOneAndUpdate`) مع (`upsert: true`) عشان لو الإعدادات مش موجودة تتعمل تلقائيًا أول مرة.

🔧 Phase 10: الجودة والتجهيز للإنتاج (Quality & Deployment)

Task 10.1: معالجة الأخطاء بشكل موحد

☐ عمل Global Error Handler middleware

☐ توحيد شكل ال Response (success/error) في كل ال API

💡 تلميح: اعمل class مخصص (`AppError`) و middleware أخير في السلسلة يمسك أي error ويرجعه بشكل موحد بدل try/catch متكرر في كل مكان.

Task 10.2: التحقق من صحة البيانات (Validation)

☐ استخدام مكتبة زي (`express-validator`) أو (`Joi`) على كل input

💡 تلميح: متعتمدش على ال validation ال Mongoose بس، ضيف طبقة validation في ال Middleware قبل ما توصل لل Controller أصلاً.

Task 10.3: كتابة توثيق للـ API

☐ توثيق كل ال Endpoints (يفضل باستخدام Postman Collection أو Swagger)

تلميح: ابدأ بـ Postman من أول Endpoint تعمله، هيووفرلك وقت كبير في الاختبار والتوثيق مع بعض.

Task 10.4: رفع المشروع (Deployment)

☐ رفع الـ Backend على منصة زي Render أو Railway

☐ استخدام MongoDB Atlas للداتايز بدل الـ Local

تلميح: تأكد إن كل الـ Environment Variables متطبطة على المنصة زي ما هي في `env.` المحلي عندك.

✓ نصيحة عامة

اشتغل Module Module بالترتيب، وبعد كل Phase اعمل اختبار كامل بـ Postman قبل ما تنتقل للي بعده. متستعجلش على الـ Frontend أو الـ Deployment قبل ما تتأكد إن الـ Core APIs شغالة ومطبوطة 100%.

بالتوفيق إن شاء الله 🚀